

# CONCEPTOS BÁSICOS DE COMUNICACIONES EN RED

## Arquitectura cliente/servidor

En esta arquitectura, un proceso central, llamado servidor, ofrece una serie de servicios a uno o más procesos clientes. En proceso servidor debe estar alojado en una máquina accesible en la red y conocida por los clientes. Cuando un cliente requiere sus servicios, se conecta con el servidor, iniciando el proceso de comunicación.

- **Los clientes**, que llevan a cabo todo el proceso de interacción con el usuario, inician los procesos de comunicación con los servidores y realizan las operaciones adecuadas para solicitarles a éstos ciertos servicios presentando al usuario los resultados de la ejecución de los mismos. La mayor parte del código se encuentra dedicado a la visualización de la información al usuario.
- **Los servidores**, que escuchan solicitudes de conexión de clientes e implementan todos los servicios que son capaces de proporcionar a los mismos, devolviéndoles los resultados de su ejecución. Generalmente no disponen de interfaz gráfica pero sí van acompañados de programas que facilitan su gestión, administración y que permiten llevar a cabo estudios sobre el rendimiento del servidor.

## TCP/IP

TCP/IP es una familia de protocolos desarrollados para permitir la comunicación entre cualquier par de ordenadores de cualquier red. Tiene 4 capas o niveles de abstracción:

- **Capa de aplicación:** en este nivel se encuentran las aplicaciones disponibles para los usuarios. Por ejemplo: FTP, SMTP, Telnet, SSH, HTTP, POP3, DHCP, DNS...
- **Capa de transporte:** suministra a las aplicaciones servicio de comunicaciones extremo a extremo utilizando 2 tipos de protocolos: TCP (Transmission Control Protocol) y UDP (User Datagram Protocol).
- **Capa de red:** tiene como propósito seleccionar la mejor ruta para enviar paquetes por la red. El protocolo principal que funciona en esta capa es IP (Internet Protocol).
- **Capa de enlace:** es la interfaz con la red real. Recibe los datagramas de la capa de red y los transmite al hardware de la red. Ejemplos: Ethernet, Token Ring, FDDI...

Los equipos conectados a Internet se comunican entre sí utilizando el protocolo TCP o UDP. Cuando se escriben programas Java que se comunican a través de la red, se está programando en la capa de aplicación. Normalmente, no es necesario preocuparse por las capas TCP y UDP; en su lugar se pueden utilizar las clases del **paquete java.net**. Sin embargo, existen ciertas diferencias entre TCP y UDP que conviene saber.

- **TCP:** Protocolo basado en la conexión, garantiza que los datos enviados desde un extremo de la conexión llegan al otro extremo y en el mismo orden que fueron enviados. De lo contrario, se notifica un error.
- **UDP:** No está basado en la conexión. Envía paquetes de datos independientes, denominados datagramas. El orden de entrega no es importante y no se garantiza la recepción de los paquetes enviados.

## Dirección IP

Una dirección IP es un número de 32 bits (IPv4) o 128 bits (IPv6) que identifica de forma única a una máquina dentro de una red que utiliza el protocolo IP.

Para saber cuál es la dirección IP de nuestro ordenador se utiliza el comando *ipconfig* (en Windows) o *ifconfig* (en Ubuntu).

## Puertos

Gracias a la dirección IP es posible enviar un paquete de datos hacia una máquina destino y efectuar la entrega correctamente. Ahora bien, en dicha máquina puede haber una gran cantidad de procesos efectuando tareas y esperando recibir paquetes de datos. ¿Cómo es posible determinar cuál de los procesos es el destinatario de los datos?

La solución la aportan los *números de puerto*. Cada proceso que lleva a cabo tareas de comunicaciones en una máquina tiene asociado un número de puerto para identificarlo. En un entero positivo de 16 bits, por lo tanto en el rango de 0 a 65535.

Cuando un cliente se debe conectar a un proceso servidor debe conocer el número de puerto del mismo, es por ello que se ha acordado mantener un rango de números de puertos reservados y fijos (hasta el 1000 aproximadamente) para los procesos servidores más comunes:

| Servidor      | Puerto |
|---------------|--------|
| SMTP          | 25     |
| POP3          | 110    |
| HTTP          | 80     |
| FTP (Datos)   | 20     |
| FTP (Control) | 21     |

# SOCKETS

La comunicación entre procesos consiste en la transmisión de un mensaje entre un conector de un proceso y un conector de otro proceso, a este conector es a lo que llamamos socket.

La programación de sockets proporciona un mecanismo de muy bajo nivel para la comunicación e intercambio de datos entre dos ordenadores, uno considerado cliente, que es el que inicia la conexión con el otro y le pide datos y el otro, servidor, que está a la espera de conexiones de clientes.

En Java, para trabajar con sockets TCP, tenemos el paquete `java.net` que nos proporciona las clases **ServerSocket** y **Socket**.

## Clase ServerSocket

Se utiliza para implementar el extremo de la conexión que corresponde al servidor, donde se crea un conector en el puerto de servidor que escucha las peticiones de conexión de los clientes.

<http://docs.oracle.com/javase/7/docs/api/java/net/ServerSocket.html>

## Clase Socket

Se utiliza para implementar el extremo de la conexión que corresponde al cliente, donde se crea un conector al host servidor al puerto que escucha.

<http://docs.oracle.com/javase/7/docs/api/java/net/Socket.html>

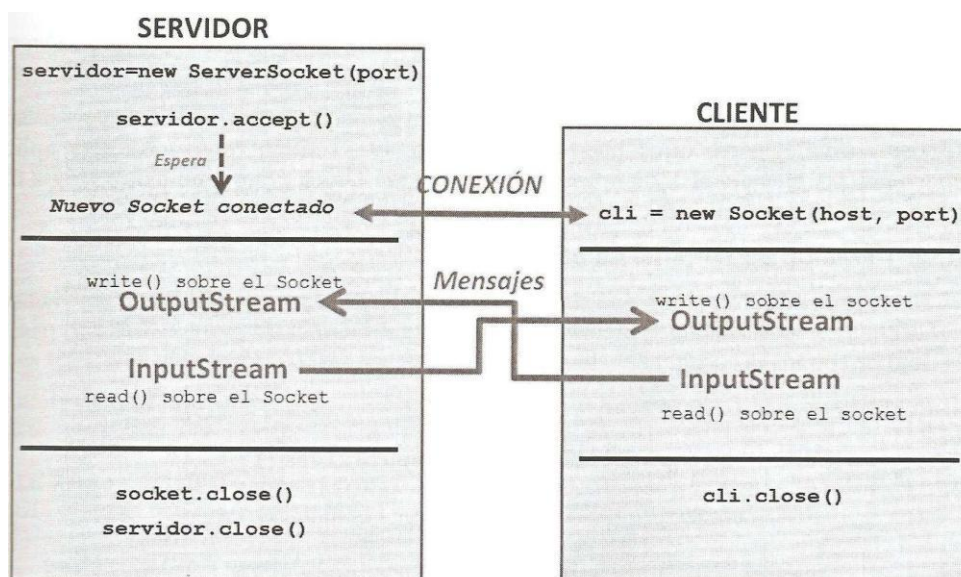
A continuación se muestra el modelo de sockets más simple:

El programa servidor crea un socket de servidor definiendo un puerto, mediante el constructor `ServerSocket(port)`, y espera mediante el método `accept()` a que el cliente solicite la conexión.

Cuando el cliente solicita una conexión, el servidor abrirá la conexión al socket con el método `accept()`.

El cliente establece una conexión con la máquina host a través del puerto especificado mediante el constructor `Socket(host, port)`.

El cliente y el servidor se comunican con manejadores **InputStream** y **OutputStream**. El cliente escribe los mensajes en el **OutputStream** asociado al socket y el servidor leerá los mensajes del cliente de **InputStream**. Igualmente, el servidor escribirá los mensajes al **OutputStream** y el cliente los leerá del **InputStream**.



## Pasos básicos en la programación de sockets en el cliente

1. **Creación del socket.** Se debe conocer la dirección IP del proceso servidor y el puerto por el que esté escuchando. Con este método el cliente obtiene el socket con el que podrá comunicarse con el servidor.

```
Socket cli;  
cli = new Socket("direccionIP", numeroPuerto);
```

## 2. Creación de los streams de entrada y de salida sobre el socket.

Se puede utilizar la clase *ObjectInputStream* para crear un stream de entrada que esté dispuesto a recibir todas las respuestas que el servidor le envíe. Esta clase permite la lectura de objetos a través del socket.

<http://docs.oracle.com/javase/1.4.2/docs/api/java/io/ObjectInputStream.html>

Con el método *getInputStream()* de la clase *Socket* se obtiene el stream de entrada para ese socket.

```
ObjectInputStream entrada = new ObjectInputStream(cli.getInputStream());
```

Para la creación de un stream de salida para enviar información al socket del servidor se puede utilizar la clase *ObjectOutputStream*.

<http://docs.oracle.com/javase/1.4.2/docs/api/java/io/ObjectOutputStream.html>

Con el método *getOutputStream()* de la clase *Socket* se obtiene el stream de salida para ese socket.

```
ObjectOutputStream salida = new ObjectOutputStream(cli.getOutputStream());
```

## 3. Leer y escribir a través del socket.

Para la clase *ObjectInputStream* el método utilizado es *readObject()*.

Para la clase *ObjectOutputStream* el método utilizado es *writeObject()*.

## 4. Cerrar los streams y el socket.

```
entrada.close();  
salida.close();  
cli.close();
```

## Pasos básicos en la programación de sockets en el servidor

### 1. Apertura del socket servidor. Hay que indicar el número de puerto donde el servidor está escuchando nuevas peticiones de conexión por parte de los clientes.

```
ServerSocket ss = new ServerSocket(numeroPuerto);
```

### 2. Esperar solicitud de conexión.

El método *accept()* bloquea al proceso servidor esperando por una conexión por parte del proceso cliente. Lo normal es que se cree un thread para atender esa conexión, mientras el servidor puede continuar escuchando nuevas peticiones de clientes a través del *ServerSocket* (lo veremos más adelante).

```
Socket clienteSocket = ss.accept();
```

### 3. Apertura del socket para la conexión.

Cuando llega una petición de conexión, se crea un nuevo socket dentro del proceso servidor (misma línea de código del paso anterior).

### 4. Creación de los streams de entrada y de salida sobre el socket.

**MUY IMPORTANTE:**

- El orden de creación de los streams debe ser:

- o `ObjectOutputStream salida = new ObjectOutputStream(cli.getOutputStream());`
- o `ObjectInputStream entrada = new ObjectInputStream(cli.getInputStream());`

- En la creación de *ObjectInputStream/ObjectOutputStream*, se intercambia un **header de serialización** entre cliente y servidor que ambos lados deben enviarse antes de poder leer/escribir objetos.

**Por este motivo si la comunicación será solo en un sentido SOLO se creará un Stream en cada lado.**

| Servidor                                                                                                                                                          | Cliente                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <pre> ObjectOutputStream salida = new ObjectOutputStream(cli.getOutputStream());  ObjectInputStream entrada = new ObjectInputStream(cli.getInputStream()); </pre> | <pre> ObjectInputStream entrada = new ObjectInputStream(cli.getInputStream()); </pre> |

5. Leer y escribir a través del socket.
6. Cerrar los streams y los sockets.

## ENVIO DE CLASES DEFINIDAS POR EL USUARIO MEDIANTE SOCKETS

Podemos enviar y recibir objetos a través de la red utilizando las clases *ObjectInputStream* y *ObjectOutputStream* siempre y cuando estos sean instancias de clases que implementan la interface *Serializable*.

La interface *Serializable* no define ningún método, solo tiene sentido semántico.

La clase *String* implementa la interfaz *Serializable* y por eso también se puede enviar de esta forma. Nuestras propias clases también se podrán enviar si implementan dicha interface. Hay que tener en cuenta que si Servidor y Cliente van a utilizar una clase definida por el usuario, esta clase debe encontrarse en ambos (la clase tiene que tener el mismo nombre y mismo paquete en ambos host). Si al leer el objeto mediante el método *readObject()* no se encuentra la clase a la que pertenece dicho objeto, el método lanza la excepción *ClassNotFoundException*.